

Crafting Interpreters VM

L0 Essence: 语言解释器与虚拟机的本质是通过词法/语法分析 (Pratt 解析) 将高层代码转化为低级字节码序列 (OPCODE)，利用栈式计算模型与寄存器/调用栈帧驱动译码循环，辅以三色标记 GC 自动化控制周期内堆内存的周转。

[M1.1] 词法扫描器 (Scanner) 与正则表达式转换规则

Scanner 逐字符扫描源码，利用双指针维护当前 Token 的物理边界。通过条件分支 (Switch-Case) 识别单字符及双字符操作符 (如 +=)，利用哈希表快速匹配关键字，将源字符流转换为包含类型、字面量与行号的 Token 列表。

[M1.2] 递归下降解析 (Recursive Descent Parsing) 与 LL(1) 冲突消除

语法分析器基于上下文无关文法 (CFG)，从最高优先级的逻辑赋值算符开始自顶向下递归解析。通过将左递归转换为迭代循环，消除多算符优先级冲突，实现无需复杂回溯的单向 LL(1) 解析流程。

[M1.3] 抽象语法树 (AST) 节点设计与 Visitor 访问者模式

解析器输出 AST 以树形表达语法结构。通过 Visitor 模式，AST 节点基类提供 accept(Visitor) 抽象接口，具体的表达式子类实现此接口以反向分发给解释器的 visit()。该设计彻底解耦了 AST 语法节点定义与后期的执行求值逻辑。

[M1.4] 动态作用域环境 (Environment) 与变量查找机制

变量环境 (Environment) 采用哈希表存储变量名与运行时值的映射。在处理嵌套块作用域时，新 Environment 内部持有一个指向外层父环境的指针 (enclosing)。在变量解析和读取时，解释器沿着此指针链向上回溯完成词法作用域变量检索。

[M1.5] Tree-Walk Interpreter 树形解释的执行限制与性能瓶颈

jlox 解释器直接采用 Tree-Walk 模式：在执行期递归地在 AST 上调用各节点的 Visitor 方法。由于每次变量获取、加减运算和逻辑判定都涉及 Java 等高级语言的多态动态分发及频繁的宿主内存对象分配，执行开销与内存分配极其沉重。

[M2.1] 字节码指令集 (OPCODE) 设计与 Chunk 存储结构

为追求性能，clox 将高级代码编译为紧凑的字节码。每个基本操作表示为一个单字节操作码 (Opcode)。虚拟机将包含字节码数组 (Code)、常量池 (ValueArray) 和行号高度压缩调试信息的结构体定义为 Chunk，作为编译与运行的流式载体。

[M2.2] 栈式虚拟机 (Stack-based VM) 计算模型与指令译码循环

clox 采用栈式 VM 模型，用一个连续的静态 Value 数组维护操作数栈 (Operand Stack)。指令指针 (ip) 指向当前字节码。译码循环利用巨大的 for(;;) 配合 READ_BYTE() 提取指令，使用 switch 分发执行栈顶数据的压入与弹出计算。

[M2.3] 虚拟机运行时值表示：C 语言 union 与 NaN Tagging 优化

VM 的基本类型通过标记联合体 Value 表达。高级优化采用 NaN Tagging 技术：在 64 位双精度浮点数中，根据 IEEE 754 标准，任何指数位全为 1 且高位分数非零的值即表示 NaN。利用这些未定义的 51 位空闲比特包装指针、布尔值及对象类型，将 Value 压缩至 8 字节。

[M2.4] 虚拟机底层调试：反汇编器 (Disassembler) 与打印栈回溯

反汇编器通过读取 Chunk 字节码并将其翻译为可读汇编指令 (如 OP_RETURN)。在调试模式下，每次译码循环执行指令前，VM 都会打印当前的计算栈元素与已反汇编的 IP 地址，直观展现指令对操作数栈的修改历史。

[M3.1] 普拉特解析法 (Pratt Parsing) 核心逻辑与结合律解析

Pratt 解析法是手写表达式编译器的高效机制。每个 Token 关联一个绑定优先级 (Binding Power)。解析器维护一个规则表，在消费 Token 时，循环比对后继 Token 的优先级，当后继优先级大于当前设定阈值时，自动结合，优雅处理了二元算符的左右结合律。

[M3.2] 缀中与缀前解析规则函数 (Prefix & Infix Parse Functions)

规则表将 Token 映射到两类编译函数：缀前 (prefix) 函数处理前缀操作符、括号和字面量 (如数字)；缀中 (infix) 函数处理双目运算符和函数调用。这种基于函数指针数组的映射分发消除了传统的嵌套 if-else 分支。

[M3.3] 普拉特解析器优先级表与单运算符分支映射

优先级从低到高定义为:PREC_NONE(最低)→PREC_ASSIGNMENT→PREC_COMPARISON→PREC_TERM(加减)→PREC_FACTOR(乘除)→PREC_UNARY→PREC_PRIMARY。核心函数 parsePrecedence(precedence) 根据当前传入优先级，向下推进编译。

[M3.4] Pratt 优先级编译示例流：‘1 + 2 * 3’ 的语法重组过程

当编译器执行 parsePrecedence(PREC_ASSIGNMENT)：

- 解析 1，调用其 prefix 函数 number() 发射 OP_CONSTANT 1。
- 遇到 +，其优先级 PREC_TERM 大于当前，调用其 infix 函数 binary()，并传入优先级 PREC_TERM。
- 递归中解析 2 发射常量，后继遇到 *，其优先级 PREC_FACTOR 高于 PREC_TERM，循环继续，递归调用 binary() 发射 3 与 OP_MULTIPLY，最终回退发射 OP_ADD。

L1 Logical Framework

- 词法解析与树形建模:** 利用 Scanner 自动状态转换生成 Token 流，基于 Visitor 模式构建抽象语法树 (AST)，提供面向行为解耦的动态树求值解释器基底。
- 字节码 Chunk 与指令译码:** 通过 Pratt 解析算符优先级并将表达式重组编译为线性 Chunk 字节码，使用 IP 寄存器在 switch-loop 译码循环中执行栈式 Push/Pop 计算。
- 栈帧闭包与上值转换:** 在 VM 计算栈上实现 CallFrame 函数调用，利用 Open 转向 Closed 的 Upvalue 结构体，打通静态闭包作用域与栈式生命周期的壁垒。
- 三色拓扑扫描与白消清理:** 由分配翻倍公式触发三色 GC，借助 Gray Stack 拓扑网罗并黑化活跃堆引用，弱引用消除后，通过 Sweep 阶段彻底清退白色死亡对象。

L2 Data Flow Topology

```
graph LR
    S[Source String] --> STS[Scanner Token Stream]
    STS --> PC[Pratt Compile]
    PC --> BC[Bytecode]
    BC --> C[Chunk]
    C --> VM[VM Run ip++]
    VM --> RI[READ_INSTRUCTION]
    RI --> SW[Switch OPCODE]
    SW --> PP[Push/Pop]
    PP --> OS[Operand Stack]
    OS --> HA[Heap Allocation]
    HA --> T[Triggers GC]
    T --> GC[GC allocated > nextGC]
    GC --> MR[Mark Roots to Gray]
    MR --> PG[Pop Gray & Push Children]
    PG --> B[Blacken active]
    B --> objects
    objects --> SWEEP[Sweep remaining white objects]
    SWEEP --> FM[Free Memory]
```

Trade-off Matrix: Compiler & Execution

▮设计维度 (Design Dimension): 方案 A (Approach A) → 方案 B (Approach B) [折衷折度 (Trade-off Balance)]

▮执行模型 vs 运行效率: Tree-Walk Interpreter (jlox) → 字节码虚拟机 (clox) [jlox 采用 AST 节点 Visitor 模式直接在节点上解释求值，极其易于实现和调试 → 但每次操作均涉及多态分发和频繁宿主内存分配，执行极为缓慢且内存膨胀严重 (执行缓慢，内存消耗巨大)]

▮虚拟机数据布局 vs 译码开销: C 语言标记联合体 (Tagged Union) → 浮点数 NaN Tagging 优化 [Tagged Union 结构清晰，便于类型扩展 → 但 NaN Tagging 通过 IEEE 754 浮点数未用比特深度压缩，将 Value 限制在 8 字节内，极大地提升了 CPU 缓存命中率 [降低内存占用，但位运算译码开销略增]]

▮表达式解析 vs 分析器体积: 传统的递归下降分析法 (Recursive Descent) → 普拉特解析法 (Pratt Parsing) [递归下降结构符合直觉 → 但 Pratt Parsing 引入优先级表和单循环驱动，以极小的代码体积优雅取代了多层语法结构函数，极大地缩小了编译器前期的分发开销 [提升前驱内聚度，解析函数大大减少]]

▮垃圾回收策略 vs 停顿延迟: 三色标记清除 GC (Mark-Sweep) → 引用计数垃圾回收 (Reference Counting) [Mark-Sweep 易于处理循环引用且分配开销低 → 但在收集期间必须执行 STW 停顿，适合轻量级 VM；若需要零延迟则必须使用带并发屏障的高级回收器 [消除复杂屏障，牺牲停顿实时性]]

[M4.1] 运行时堆内存管理与动态类型 Tag 编解码

所有生命周期超出单次调用栈的复杂对象（Obj）均在堆中动态分配。所有 Obj 派生类（如 ObjString）的头部均包含一个统一的 Obj 结构体，内含垃圾回收的可达性标记与类型枚举标签。运行时通过此标签判定具体子类型并安全向下转型。

[M4.2] 自研哈希表设计：开放寻址与线性探测（Linear Probing）

clox 自研哈希表采用线性探测方式解决冲突：`index = (hash + 1) % capacity`。数据项紧凑保存在单一连续数组中，提升 CPU 缓存命中率。当表的负载因子超过 75% 时触发扩容，分配双倍容量空间并把所有存活键值对重新 Hash 重组。

[M4.3] 哈希表软删除：墓碑标记（Tombstone）的作用与判定

开放寻址哈希表在物理删除元素时不能直接置空，否则会导致线性探测链断开。删除操作会将该槽位修改为墓碑标记（Tombstone）。查找时遇到墓碑会继续探测后续槽位以防断链；插入时则会优先复用墓碑槽位，控制表的物理膨胀。

[M4.4] 字符串驻留（String Interning）机制与快速等值判定

VM 维护一个全局的字符串驻留哈希表（Intern Table）。在堆上分配新 ObjString 前，必须首先在该表中检索是否存在字面量一致的字符串。如果存在则复用单例指针，如果不存在才执行物理分配并存表。由此将复杂的 $O(N)$ 字符串内容比对降维为 $O(1)$ 指针比对。

[M5.1] 分支与循环字节码生成：回填技术（Backpatching）

编译器编译 `if` 时，先发射跳转指令（OP_JUMP）并填充两个字节的占位占空偏移。随后编译 `then` 分支，等 `then` 分支全部编译完毕，根据生成的物理字节长度计算出实际要跳转的指令偏移，并“回填”到先前的占位符中。

[M5.2] 闭包设计：Upvalue 上值概念与静态作用域捕捉

当嵌套函数引用外部函数的局部变量时，该变量在编译期被识别为 Upvalue。编译器维护一个编译期数组记录上值信息。运行时，函数被实例化为 ObjClosure，内部持有一个指向真实局部变量的 ObjUpvalue 指针数组，用于执行闭包求值。

[M5.3] 开放与闭合上值（Open vs Closed Upvalues）在 VM 栈中的状态转换

- **Open 状态**: 当被捕捉的外部局部变量仍处于活动栈帧内时，ObjUpvalue.location 直接指向该变量在 VM 计算栈上的物理地址。
- **Closed 状态**: 当外部函数返回、对应栈帧销毁时，VM 遍历开放上值链表，将栈上的数据拷贝到 Upvalue 自身的 C 堆内存中，并将 location 指向自身的 value 域，转换为 Closed。

[M5.4] 面向对象编程：类定义、实例属性与方法分发机制

ObjClass 结构体包含一个存储方法的哈希表。实例化时，VM 动态分配 ObjInstance 并关联该类，实例内部维护自身的属性字段哈希表。方法调用时，VM 查找类的方法表，将其与实例 `this` 指针打包为 ObjBoundMethod 压入操作数栈。

[M5.5] 超类继承（Superclass Inheritance）与 Method Bindings

编译类继承时，派生类首先深度复制超类的方法表。当在子类内部编译 `super.method()` 调用时，编译器静态解析超类位置，并在运行期执行动态 Method Binding，将超类方法与当前实例的引用强行绑定，确保正确的虚方法分发。

[M6.1] 根对象扫描（Marking Roots）与 GC 触发阈值公式

GC 由堆内存分配翻倍公式触发：当已分配字节数超过下一阶段阈值 `allocatedBytes > nextGC` 时启动垃圾回收。GC 首先扫描根对象（Roots），包括当前操作数栈、活跃 CallFrame 的函数帧、全局变量表以及活跃编译器，将其全部涂为灰色。

[M6.2] 三色标记机制：白色、灰色与黑色对象的含义

- **白色**: 待回收的垃圾对象。在 GC 开始时所有堆对象默认为白色；GC 结束后保持白色的对象将被物理清除。
- **灰色**: 活跃且已被 GC 根扫描到，但其内部持有的子对象引用（如闭包的上值、实例的方法）尚未被完全扫描的对象。
- **黑色**: 自身是活对象，且其持有的所有子对象引用也已完全扫描标灰的对象。

[M6.3] 灰色对象工作队列（Gray Stack）维护与溢出防范

虚拟机内部维护一个动态增长的灰色对象指针数组（Gray Stack）作为 GC 工作队列。发现灰色对象时压入该栈。为防范因极端物理内存耗尽导致 Gray Stack 分配失败，GC 引入溢出逃逸保护：此时退回遍历所有堆对象将可达的标记染灰。

[M6.4] 引用网推导扫描（Tracing References）与黑化过程

工作循环持续弹出 Gray Stack 栈顶的对象指针，将其由灰色染为黑色（Blacken）。随后读取该对象内部的所有子成员引用，若子引用对象为白色，则将其染为灰色并重新推入 Gray Stack 压栈，直到灰色栈彻底清空。

[M6.5] 弱引用消除：字符串驻留哈希表（Intern Table）的 GC 回收

全局 String Intern 表存储的指针不视为根对象引用以避免泄露。在 Mark 阶段结束后、Sweep 开始前，GC 必须调用 `tableRemoveWhite()` 过滤该表。扫描整个 Intern 表，主动清空残留的白色（未标记）字符串键值对。

[M6.6] 清扫阶段（Sweeping Phase）物理内存回收与堆链表整理

清扫器物理遍历堆上挂载的全局对象链表。对比每个对象的 GC 标记：若标记为白色，说明已不可达，将其从堆链表中摘除，并调用 `freeObject()` 释放其物理 C 内存；若为黑色，说明仍存活，将其重置为白色并清空 GC 标记，为下一次 GC 循环做准备。

Zone T1: Opcode & Parameters

Zone T2: VM Diagnostics & Debugging

`vm.stack` 打印栈帧：单步跟踪 VM 时，循环遍历 `vm.stackTop` 之前的元素，打印 `operand`

`stack` 的具体对象以排查压栈出栈逻辑。`\disassembleInstruction(chunk, offset)` 反

汇编：读取 `Chunk` 字节码数组，针对不同指令打印相应的物理偏移量、操作数并计算下

一个指令的偏移。`\tableRemoveWhite(table)` 弱引用扫描：在标记阶段结束后、清扫

开始前，对全局字符串驻留表（Intern Table）执行弱引用过滤，回收未被标记的字符串。

`\freeObject(obj)` 物理内存释放：在 Sweep 阶段针对白色 Obj 执行物理 `reallocate` 内

存释放，解构 Obj 对应的特定数据内存域。